



University
of Glasgow | School of
Computing Science

Honours Individual Project Dissertation

EXPLORING GENERATIVE DEVELOPMENT WITH mBOT

Alexandra Markopoulou
August 2, 2024

Abstract

Higher-level Generative Development tools have risen in popularity, rapidly expanding use cases. Despite this fact, such tools are lacking supporting infrastructure in educational environments involving off-the-shelf educational robots, such as the mBot by Makeblock. The aim of this project is to address this issue by building a framework to explore the use of State Machines and Reactive Systems for the mBot. This was a challenging task since Makeblock restricts development to their proprietary software. It was found that it is possible to establish connections between higher-level Generative Development tools and the mBot, with the potential to extend the use of such tools to other educational robots.

Education Use Consent

I hereby grant my permission for this project to be stored, distributed and shown to other University of Glasgow students and staff for educational purposes. **Please note that you are under no obligation to sign this declaration, but doing so would help future students.**

Signature: Alexandra Markopoulou Date: August 2, 2024

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Aim	1
1.3	Outline	2
2	Background and Related Work	3
2.1	The mBot Robot	3
2.2	Generative Development Overview	4
2.2.1	Experiments with an Arduino Uno Custom Made Robot	4
2.2.2	State Machines	5
2.2.3	UMPLE	5
2.2.4	Reactive Systems	6
2.2.5	Spectra Synthesis	7
2.3	Serial Port Communication	7
3	Configuring Communication between MBot and a Computer	9
3.1	Controlling mBot with C++ Firmware	9
3.2	Choosing the Appropriate Java Library	11
3.3	Configuring 2-Way Communication	11
3.4	Ultrasonic Sensor Data Transmission	13
3.5	Contribution	14
4	Configuring mBot with UMPLE	15
4.1	Turning the mBot into a State Machine	15
4.2	UMPLE Code	15
4.3	UML Diagram of UMPLE Implementation	17
4.4	Communication between mBot and UMPLE	19
4.5	Contribution	21
5	Reactive Synthesis for mBot with Spectra	22
5.1	mBot as a Reactive System	22
5.2	Spectra Specification	22
5.3	UML Diagram of Spectra Implementation	24
5.4	Spectra Controller	24
5.5	Communication between mBot Sensors and Spectra	25
5.6	Evaluation of the Spectra Specification	26
5.6.1	Navigation	26

5.6.2	Sensor Output and mBot Response	28
5.7	Contribution	28
6	Initial Steps in WiFi Module Integration	30
6.1	Circuit Schematic	30
6.1.1	ESP8226 - D1 Mini Pin Diagram	30
6.1.2	Voltage Divider	31
6.2	Software Implementation	31
6.3	Contribution	32
7	Further Discussion	33
7.1	Conceptual Evaluation	33
7.2	Challenges	33
7.2.1	Working with the mBot Hardware	33
7.2.2	Working with Makeblock Proprietary Software	34
8	Conclusion	35
8.1	Summary	35
8.2	Reflection	35
8.3	Future Work	36
Appendices		37
A	Appendices	37
A.1	mCore PCB Schematic	37
A.2	Complete UML Diagram of the Java Application	37
Bibliography		39

1 | Introduction

Generative development (GenDev) has emerged as a powerful approach in software engineering, allowing for the automatic generation of software based on high-level specifications. This methodology has found widespread application in a plethora of domains, allowing for the development of complex systems with reduced manual effort.

One area where GenDev is gaining traction is robotics, where it has the potential to streamline the development process and improve the capabilities of robots. Additionally, in the past decade, we have also seen a rise in off-the-shelf educational robots, such as the mBot from Makeblock or Sphero Bolt from Sphero¹. Such devices represent an exciting opportunity to popularize GenDev methods, making them more accessible to a broader audience, including students and enthusiasts with limited electronics knowledge.

1.1 Motivation

The motivation for this project stems from the initiative to broaden GenDev's accessibility to students and enthusiasts, particularly in the field of robotics. The mBot robot is an excellent platform for this goal, providing a user-friendly and widely available hardware solution. We hope to lower the barrier to GenDev experimentation by creating infrastructure that is compatible with Arduino Uno-based systems such as the mCore.

We want to explore different approaches to GenDev regarding robotics and facilitate experimentation with state machines and reactive systems by using tools like UMPLE and Spectra. UMPLE is a tool which allows users to design high-level abstractions and generate code from them. Spectra is a formal specification language specifically tailored for use in the context of reactive synthesis, an automated procedure to obtain a correct-by-construction reactive system from its temporal logic specification.² By integrating tools such as UMPLE and Spectra Synthesis into the mBot, we hope to encourage users to experiment with different GenDev techniques and methodologies.

Currently, there is insufficient infrastructure to support GenDev on platforms such as the mBot, necessitating the creation of specialised frameworks.

1.2 Aim

The aims of this project are as follows:

- Develop a comprehensive framework for enabling GenDev on Arduino Uno-based devices.
- Integrate the mBot with UMPLE and Spectra Synthesis to facilitate experimentation with GenDev principles.
- Evaluate the application of these methods of GenDev and their applicability in the context of educational robotics.

¹<https://sphero.com/>

²<https://smlab.cs.tau.ac.il/syntech/spectra/>

- Ensure compatibility and ease of replication of the framework across other Arduino Uno-based devices.

1.3 Outline

The structure of this project follows the design process of the firmware to allow the mBot to be programmed with higher-level tools. Initially, background knowledge is presented to allow the reader to follow the design process. Next, a generic communication firmware is developed in Chapter 2, which is further altered in Chapters 3 and 4 depending on the requirements of each high-level tool. Lastly, a reflection and further discussion of the project are presented in Chapters 6 and 7.

- **Chapter 2 - Background and Related Work**
This chapter discusses related work and background knowledge needed.
- **Chapter 3 - Configuring Communication between MBot and a Computer**
This chapter presents how the firmware to establish a connection between the mBot and a computer is designed and implemented.
- **Chapter 4 - Configuring mBot with UMPLE**
This chapter presents how the mBot is configured to behave as a state machine using UMPLE
- **Chapter 5 - Reactive Synthesis for mBot with Spectra**
This chapter presents how the mBot is elevated as a reactive system using Spectra
- **Chapter 6 - Initial Steps in WiFi Module Integration**
This chapter discusses how to integrate a third party Wi-Fi module on the mBot
- **Chapter 7 - Further Discussion**
This chapter discusses the challenges encountered during this project and evaluates the concept of using higher-level GenDev tools with educational robots.

2 | Background and Related Work

2.1 The mBot Robot

The mBot, as seen in figure 2.1, is an educational robot kit, produced by Makeblock¹, developed to familiarize students with computer science, electronics and robotics. It is crafted as a hands-on learning tool, aiming to make the worlds of coding and robotics accessible and engaging for learners.



Figure 2.1: The mBot

The most important hardware components of the mBot are the mCore, as seen in Figure 2.2, which is based on an Arduino-Uno microcontroller, Bluetooth communication, DC Motors and Wheels to move, an Ultrasonic sensor to measure distance in front of it and Infrared Line-Following sensors, which allow the mBot to navigate predefined paths. As Jelena Pisarov has demonstrated, these hardware components serve as a great introduction for students to concepts of robotics and electronics.(Pisarov 2019)

Furthermore, the mBot is intended to be programmed using Makeblock's IDE, mBlock, which only supports Python and educational coding "blocks". This IDE has limited scalability since its applications are restricted by the set of proprietary packages provided by Makeblock. The experiments described in 2.2.1 would be impossible to execute using mBlock since they require the use of C++, Java and third-party libraries which are not supported. In order to further develop the behaviour of the mBot, this software limitation needs to be overcome.

Hence, we need a different method to approach programming the mBot. Fortunately, since the mCore 2.2, mBot's electronic board, is based on Arduino Uno, it is possible to work around the software limitations as shown in section 3.

¹<https://www.makeblock.com>

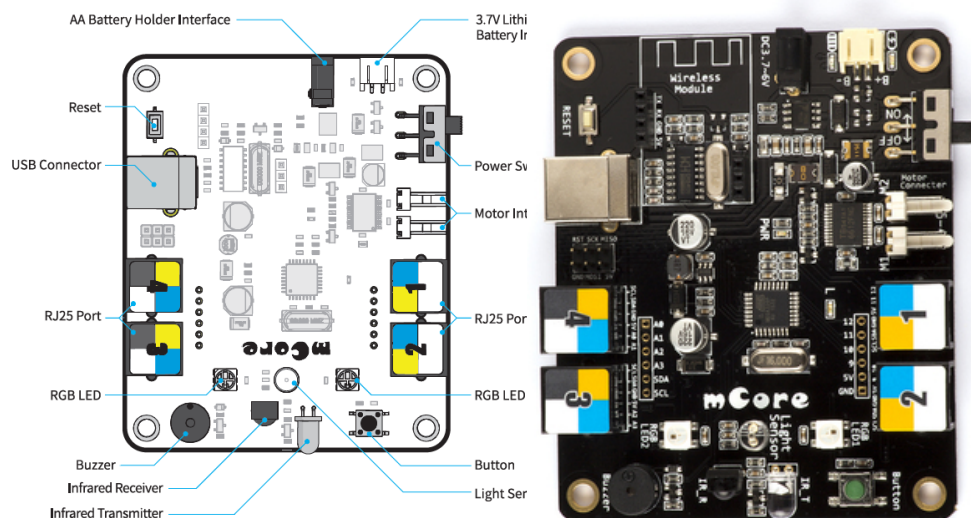


Figure 2.2: mCore - Board of the mBot (Makeblock 2021)

2.2 Generative Development Overview

Generative software development concerns creating system families through identifying commonalities and variabilities within a given problem domain. (Czarnecki 2005) In contrast to conventional methodologies that revolve around individual systems, this approach aims to automate the generation of system variants by utilising reusable assets like common architectures, components, and models. The use of Domain-Specific Languages (DSLs) is fundamental to this methodology, providing a precise and domain-focused means of expressing system specifications.

The generative process encompasses two key development phases:

1. Domain engineering, concentrating on creating reusable assets,
2. Application engineering, tasked with constructing concrete applications using these assets.

Feature modelling and a feature-oriented approach play pivotal roles, facilitating the collection and representation of common and variable features within system families. (Czarnecki 2005)

While closely aligned with model-driven development, generative software development stands out by distinctly emphasizing the generation of system families, making it a potent strategy for achieving systematic software reuse and effectively managing variability in intricate problem domains. (Czarnecki 2005)

In this section, we first give an overview of generative development experiments with an Arduino Uno custom-made robot, as presented in an online tutorial, and then we give an overview of each of the concepts, tools and models, used in these experiments relevant to our work, such as state machines and UMPLE, reactive systems and the Spectra synthesiser.

2.2.1 Experiments with an Arduino Uno Custom Made Robot

A series of generative development experiments are available on Youtube² from Professor Jan Oliver Ringert(Ringert 2023). These videos served as tutorial material for the MSc module Generative Development at the University of Leicester in 2021.

²Link to the Youtube playlist by Jan Oliver Ringert.

The robot used in these experiments is called GenDevBot and was designed by Professor Jan Oliver Ringert. Its main components are an Arduino Uno board, 2 DC Motors with wheels attached to them, an Ultrasonic sensor and an ESP 8266, which is a Wi-Fi integrated circuit (Ringert 2023).

There were several experiments on generative development, amongst which we chose to focus on "*GenDevBot: Code generation from UML state machines via UMPLE for an autonomous robot*"(1) and "*GenDevBot ESP8266 Reactive Synthesis Tutorial*"(2).

The first experiment illustrates how UMPLE can be used to represent state machines. The experiment walks through the process of writing a UMPLE specification and demonstrates step-by-step how to make it more complex by incorporating additional states and transitions between them.

The second experiment demonstrates Spectra specification language and provides a tutorial on how to write basic Spectra specifications. Similarly, the process and logic of a reactive system are shown step by step, demonstrating how it can become complex and adaptive.

It should be noted that these experiments focus on the theory behind the concepts of State Machines and Reactive Systems, but not on the technical configuration between GenDevBot and Spectra or UMPLE. This project focuses on broadening the set of robots that are able to be programmed using the aforementioned tools, like GenDevBot

The mBot described in Section 2.1 will be used to replicate the above experiments. The purpose of this robot is to recreate the research that Jan Oliver Ringert carried out. Makeblock provides compatible and pre-configured hardware in the mBot. This allows for a substantial level of abstraction in running these experiments, since the user does not need to have thorough knowledge of electronics. By using this product, research can focus more on software design and analysis by removing away the complexity of hardware configuration. For the experiments that will be covered in this project, this abstraction layer can provide consistency and reproducibility.

2.2.2 State Machines

A state machine is a mathematical model used in computer science to represent the behaviour and architecture of discrete state systems, both in software and hardware.(Yodaiken 2023) They operate by reading a series of inputs and transitioning between different states based on those inputs. Each state defines which state to move to for a given input, providing a structured way to process input sequences. State machines can be deterministic, where each input from a state leads to a single state transition, or non-deterministic, where an input from a state can lead to multiple possible transitions (Shead 2018).

We will explore the logic and behaviour of State Machines in section 4. We will design a State Machine for the mBot, allowing us to explore how State Machines can effectively control the behaviour of autonomous systems.

2.2.3 UMPLE

UMPLE is a Model-Oriented programming language. Model-oriented programming uses the practice of generating code from a model (UMPLE). UMPLE is an open-source tool which allows the user to design high-level abstractions (models) and generate code in widely used programming languages. Its generative capabilities are best demonstrated in Java since all other supported languages are still in developmental stages. It supports software development by providing both a textual and visual representation of high-level abstractions, allowing the user to obtain a multifaceted perspective on their project.(Lethbridge et al. 2021)

UMPLE serves as both a compiler and a web-based front-end. It takes as input UMPLE code and can also produce state machine diagrams, in addition to generated code. Figure 2.3 illustrates an

example code for an on/off light bulb system and the resulting state machine diagram generated by UMPLE. In this case, the state machine consists of two states, `On` and `Off`, and the arrows from one to the other denote possible state transitions: for example, the active state is changed from `On` to `Off` when some button is pushed.

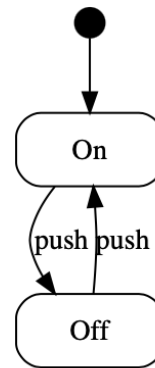
The syntax UMPLE follows for state machines is meant to be intuitive and straightforward. For example, in listing 2.1 line 3 declares the name of the state, in this case `bulbState`. Line 4 declares a new state, `On`. When the action `push` is executed, which is represented by the arrow `->`, the state changes to `Off`. Similarly, line 5 declares a state `Off` which becomes `On` when the action `On` is executed.

```

1 namespace example;
2   class lightBulb{
3     bulbState{
4       On {push -> Off;}
5       Off {push -> On;}
6     }
7   }

```

Listing (2.1) Example UMPLE Code



(a) Example UMPLE Diagram

Figure 2.3: UMPLE Example

2.2.4 Reactive Systems

A Reactive System is a software architecture based on reactive programming, which focuses on the data flow of the system. According to the book, "What Is a Reactive System?" by Peter Royal (Royal 2023), a Reactive system usually presents the following properties;

- **Responsiveness:** Quick responses to user requests, providing feedback and consistent timing. This feature is important for user satisfaction and can be implemented with internal timeouts and effective error-handling techniques.
- **Resilience:** They are resilient to failure, meaning they can continue to function even in the presence of faults or errors. Components of the system are designed to accept failures, reducing downtime and ensuring overall system stability.
- **Elasticity:** They demonstrate elasticity by dynamically adjusting the utilisation of resources to accommodate varying workloads. They can scale up or down in response to demand needs, ensuring peak performance.
- **Message-Driven Communication:** Communication between components in a reactive system is asynchronous and message-driven. This approach decouples components / promotes loose coupling and provides location transparency, allowing for more versatility in system architecture and deployment.

In summary, a reactive system is a system that reacts to stimuli from its environment. Therefore, the mBot can already be considered to be a reactive system. For example, it can be programmed to take a reading from a sensor and react to it. However, in the experiments described in section 2.2.1, the mBot will become a reactive system with even more capabilities, incorporating properties such as Message-Driven communication and Responsiveness.

2.2.5 Spectra Synthesis

The Spectra Synthesizer (Maoz and Ringert 2021) used in the experiments described in section 2.2.1 is based on Reactive Synthesis, an automated process that generates correct-by-construction reactive systems from temporal logic specifications (Baier and Katoen 2008). Unlike traditional methods involving manual implementation followed by verification, synthesis offers an automated approach to ensure correctness if a feasible implementation exists. However, there are acknowledged issues with reactive synthesis, particularly in the specification language’s syntax, semantics, and expressiveness. Spectra is a novel specification language tailored for reactive synthesis, complemented by the Spectra Tools³ suite. Spectra Tools provide a synthesizer for obtaining correct implementations, alongside analyses aiding engineers in writing high-quality specifications.

Spectra offers a user-friendly language and environment for formal specification development and experimentation. Undergraduate students have been able to develop specifications for robots and other reactive systems, as shown in section 2.2.1. In this project, we aim to develop Spectra specifications for the mBot and integrate them. As outlined, it would enable the use of a high-level reactive system specification language on an accessible system.

Listing 2.2 showcases an example Spectra Specification for a traffic light system, as presented by professor Jan Oliver Ringert (Ringert 2021). Lines 1 – 3 declare boolean environment variables, representing different conditions and events of the environment. Lines 5 – 6 declare a boolean system representing which traffic is on. Lines 8 – 9 assume (asm) that a green traffic light, meaning a value of True for the boolean variable, will always eventually (alwEv) occur. Line 11 always (alw) guarantees (gar) that traffic lights A and B cannot be green at the same time. Lines 12 – 13 guarantee that always eventually the traffic light A or B will be green (true), or that there will be an emergency. Lastly, line 15 guarantees that whenever there is an emergency, traffic lights A and B will always be red (false).

```

1  env boolean carA;
2  env boolean carB;
3  env boolean emergency;
4
5  sys boolean greenA;
6  sys boolean greenB;
7
8  asm alwEv carA;
9  asm alwEv carB;
10
11 gar alw !(greenA & greenB);
12 gar alwEv greenA | emergency;
13 gar alwEv greenB | emergency;
14
15 gar alw emergency -> (!greenA & !greenB);

```

Listing 2.2: Example spectra specification for a traffic light system

2.3 Serial Port Communication

Serial communication involves transmitting data bit by bit using a two-wire connection: a transmitter (sender) and a receiver. In this form of communication, data is transmitted sequentially,

³<http://smlab.cs.tau.ac.il/syntech/spectra/>



Figure 2.4: USB Type B to USB Type A cable

i.e. one after the other. Unlike parallel transfer, which sends multiple bits simultaneously, serial transfer makes use of a single path or cable, reducing the need for complex hardware. As digital circuit speeds have improved, serial transfer has become more practical due to its simplicity and cost-effectiveness. This has resulted in the development of various serial I/O methods and standards, which are now widely used in electronic equipment to connect devices, networks, printed circuit boards, and integrated circuit chips. (Frenzel 2016)

In this project, serial port communication will be used to establish communication between the mBot and the technologies used to complete the experiments mentioned in section 2.2.1. As aforementioned, it is a simple and effective communication protocol which is likely to be compatible with educational robots aside from the mBot, thus making our framework more robust.

However, it should be acknowledged that the possibility of overloading the serial ports is plausible. For this reason, throughout the experimentation process, the behaviours of the mBot will be time-limited to ensure a smooth transmission of data.

In this project, the hardware used is a cable with USB Type A to Type B as shown in Figure⁴ 2.4. The mBot has a type B port as shown in figure 2.2, and USB type A is used to connect to a computer. Regardless, the firmware developed is compatible with any type of cable, as long as there is a serial port connection. Lastly, the project is developed on a Macbook with macOS Ventura 13.5.2.

⁴Source of cable picture

3 | Configuring Communication between MBot and a Computer

We need to configure the mBot to be programmable with Java since this is the language that the experiments in this project are based on, as described in section 2.2.1.

Ideally, this would be a straightforward procedure, but there were several challenges raised by compatibility issues of the mBot. As aforementioned in section 2.1, the mBot is designed to seamlessly work with Makeblock's own proprietary IDE, mBlock. This IDE is not flexible enough and thus does not support the experiments we will conduct, since it solely supports Python and mBlock educational block programming.

Hence, we need to configure communication between a computer and the mBot using a different method, allowing us to work with powerful and versatile IDEs. This entails that we need to build firmware which will allow us to control the mBot from a different IDE, which is against the intended use of the device.

In this chapter, we build a C++ firmware to control the mBot, explore different Java libraries for Serial Port communication and configure a 2-way communication between the mBot and a Java Application.

3.1 Controlling mBot with C++ Firmware

As shown in section 2.1, the mBot is equipped with a board based on Arduino-Uno. Therefore, this should enable the robot to be programmable using ArduinoIDE, and IDE which supports C++. Makeblock provides a guide on how to use ArduinoIDE to program the mBot using C++, along with 3rd party drivers to access the mBot outside of its bespoke software (Team 2022). Additionally, some libraries to control the mBot's hardware are provided by Makeblock; such as motors, light sensors, sonar sensors, etc. They are discussed and presented during the experimentation process. (Makeblock 2020)

Even with the guides provided by Makeblock, configuring the mBot to work with a different IDE still proves to be challenging. If we were to use mBlock, the mBot's default IDE, all Serial-Port and Bluetooth communication would be configured by it. Since we need to use ArduinoIDE instead, it is required to manually configure the Serial Communication ports and build firmware to send instructions to the mBot. Unfortunately, using ArduinoIDE entails that Bluetooth communication cannot be used in these experiments. The reason is that Makeblock has configured the Bluetooth module to solely be accessible through their proprietary IDE, mBlock. This makes it impossible to connect to the mBot via Bluetooth outside of mBlock.

Listing 3.1 illustrates a program which controls the mBot's DC Motors using C++. Lines 1-6 import the library for the mBot by Makeblock and declare two variables that represent the two DC Motors of the mBot, left motor `MotorL` and right motor `MotorR`. Lines 8 - 24 declare the types of movement the mBot can follow. These movements are determined by the function from `MeDCMotor`, `run()` where speed and direction can be specified. For example, +100 would be going forward with a speed of 100, whereas -100 would be going backwards with a speed of 100.

However, in the function `Forward()` in lines 8-12 the left motor is given `-movespeed` and the right motor is given `+movespeed`. While experimenting with the firmware, we noticed that the left motor seemed to be behaving in the opposite direction of the instruction it received. There was no apparent cause of why it behaved this way, even after rebuilding the motor system in the mBot. Therefore, we decided to tackle this bug by providing the opposite direction of the intended outcome to the left motor.

Lastly, in lines 31-33, the `setup()` function is run once after compilation is complete. In software engineering, it is standard practice to print a message to a serial port to indicate that this process was run without any errors, hence why `Serial.println("Arduino ready!")` is used. The function `loop()` in lines 35-41 will run indefinitely after `setup()` is completed. In this example, the mBot will move forward, stop, turn, backward and then stop on repeat.

```

1  #include "MeMCore.h"
2
3  MeDCMotor MotorL(M1);
4  MeDCMotor MotorR(M2);
5
6  int movespeed = 100;
7  int movementDuration = 1000; // in milliseconds
8
9  void Forward(){
10     MotorL.run(-movespeed);
11     MotorR.run(movespeed);
12     delay(movementDuration);
13 }
14
15 void Backward(){
16     MotorL.run(movespeed);
17     MotorR.run(-movespeed);
18     delay(movementDuration);
19 }
20
21 void Turn(){
22     MotorL.run(movespeed);
23     MotorR.run(movespeed);
24     delay(movementDuration);
25 }
26
27 void Stop(){
28     MotorL.stop();
29     MotorR.stop();
30 }
31
32 void setup() {
33     Serial.println("Arduino ready!");
34 }
35
36 void loop() {
37     Forward();
38     Stop();
39     Turn();
40     Backwards();
41     Stop();
42 }

```

Listing 3.1: Firmware for controlling the movement of the MBot

3.2 Choosing the Appropriate Java Library

As outlined in section 2.6, the experiments presented require the use of Java to be completed. Yet again, the mBot does not provide any built-in functionality to allow for communication with anything but its own IDE, mBlock. Therefore, we need to build software which will allow the mBot to receive and send messages with a Java program. To successfully build this two-way communication, Serial-Port communication is used as it is already supported by the mBot and there are plenty of Java libraries available to choose from.

To overcome this incompatibility, we need to build controllers and supporting firmware for two-way communication with Java. This is essential, as the two main technologies used in the experiments, UMPLE and the Spectra Synthesizer, were created to work with Java. Without the following section, none of the experiments would be possible to conduct.

The following Java libraries were tested to discover which one would work best with the mBot:

- **JArduino.** Instead of writing code in ArduinoIDE in C++, JArduino provides the opportunity to use Java to control your Arduino board. It provides a Java API for communication with Arduino Boards, enabling interactions with sensors, actuators, and other components (Brice and et al 2018). Despite its promising capabilities, it was not compatible to be used with the mBot. The library is slightly outdated which led to version compatibility issues with the library's firmware.
- **Java Communications API.** Also known as javax.comm, this Serial Port communication API provides applications with access to Serial Ports and is an official Java Library (Oracle). However, it does not provide a lot of abstraction and can be difficult to configure correctly with the mBot.
- **jSerialComm.** This is a Java library designed to provide a platform-independent way to access standard serial ports without requiring external libraries, native code, or any other tools. It is still being maintained and updated while providing various key features such as timeout support and event-based reading/writing. This seemed to be the most suitable option for this project, and therefore it was chosen as the Java library to be used.

It should be briefly noted that serial port communication is not the optimal way of communication when using the mBot. It is inefficient when compared to wireless connection and has a higher chance of invalid data transfer due to serial port overload. However, the mBot does not provide a straightforward method to establish wireless communication. Its Bluetooth module is set to connect only with mBlock, and its proprietary Wi-Fi module has been discontinued in the UK. This will be further addressed in section 6.

3.3 Configuring 2-Way Communication

In order to establish two-way communication in Java we wrote the code in listing 3.2. The constants on lines 3–6 are port time-out time and the name of the serial port the mBot will connect to. In this example, the port on the computer used for communication with the mBot was called "cu.usbserial-14340"; however this port could be named differently in other machines. Lines 10–19 are to set up and open the port connection. Line 19 defines a single character which is sent via the serial port, in this case, 'C'. Lines 37–43 show in more detail how data is written as an output stream. Lines 22–31 receive data from the serial port by creating a buffer and reading

from it. On line 26, the condition to stop receiving data is sent, in this case receiving -1 would stop this process. This practice gives the mBot the ability to control communication, since if it sends -1 it will prevent the Java program from receiving data. Lastly, line 34 closes the port after communication has ended.

```

1  public class SerialPortExample {
2
3  // Set timeout to 5000 milliseconds (5 seconds)
4  public static int PORT_TIMEOUT = 5000;
5  //example serial port the mBot is connected to
6  public static String PORT = "cu.usbserial-14340";
7
8  public static void main(String[] args){
9
10     // Get the serial port by name
11     SerialPort serialPort = SerialPort.getCommPort(PORT);
12
13     serialPort.setComPortTimeouts(SerialPort.TIMEOUT_READ_SEMI_BLOCKING,
14                                   PORT_TIMEOUT, 0);
15
16     // Set the baud rate (9600 bps)
17     serialPort.setBaudRate(9600);
18
19     // Send Data, in this case the character C
20     sendData(serialPort, 'C');
21
22     // Receive data
23     byte[] buffer = new byte[1024];
24     int bytesRead;
25     try {
26         bytesRead = serialPort.getInputStream().read(buffer);
27         if (bytesRead != -1) {
28             String receivedData = new String(buffer, 0, bytesRead);
29         }
30     } catch (IOException e) {
31         e.printStackTrace();
32     }
33
34     // Close the serial port
35     serialPort.closePort();
36
37     public static void sendData(SerialPort serialPort, char msg) {
38         try (var outputStream = serialPort.getOutputStream()) {
39             outputStream.write(msg);
40         } catch (Exception e) {
41             e.printStackTrace();
42         }
43     }

```

Listing 3.2: Java JSerialComm program for two-way communication with the mBot connected to a serial port

Now that the foundation has been built in Java, we finalise communication with a program on ArduinoIDE in C++ shown in Listing 3.3. Lines 9-19 check if there is data incoming from

the serial port, and if so it is received and saved in the local variable `receivedByte`. Lines 14–18 check if the received character was 'C', as we set it to be on Listing 3.2. If it was 'C', then the program replied by sending `\verb'B'` and then the command to move Forward is sent to the mBot, using the method from listing 3.1.

```

1
2   void setup() {
3   // Initialize serial communication at 9600 baud
4   Serial.begin(9600);
5   }
6
7   void loop() {
8   // Check if data is available to read from the serial port
9   if (Serial.available() > 0) {
10    // Read the incoming byte
11    char receivedByte = Serial.read();
12
13    // Example: If receivedByte is 'C', send back 'B'
14    if (receivedByte == 'C') {
15        Serial.print("Sending: ");
16        Serial.write('B');
17        Forward(); // the mBot moves Forward
18    }
19  }
20 }
```

Listing 3.3: mBot C++ program for two-way communication

3.4 Ultrasonic Sensor Data Transmission

Another crucial aspect of communication between the mBot and a computer is the transmission of information gathered from its ultrasonic sensor.

For this purpose, we write the code in Listing 3.4 to allow the mBot to transmit information gathered from its ultrasonic sensor. Lines 4 and 9 signal to the serial port that a sensor reading is in process, while lines 6 and 7 measure and send the reading. As seen in line 5, some delays are added to prevent the serial port from overflowing. In this case, the delays are in milliseconds.

```

1   MeUltrasonicSensor ultraSensor(PORT_3);
2   ...
3   if (receivedChar == 'C'){
4       Serial.print("Sensor_Reading");
5       delay(300);
6       double distance = ultraSensor.distanceCm();
7       Serial.print(distance);
8       delay(300);
9       Serial.print("end_Sensor_Reading");
10      delay(100);
11  }
```

Listing 3.4: Code in C++ to take readings from the Ultrasonic Sensor

Lastly, listing 3.5 showcases how the information of a sensor reading, as shown in listing 3.4, is handled from the Java application. Lines 3 and 5 showcase how processes start and end in the system. In this case, the process for receiving and processing data from the ultrasonic sensor begins with the key phrase `Sensor_Reading` and terminates with `End_Sensor_Reading`. This feature is in place to prevent unnecessary data processing and make the system more effective, by allowing certain processes to commence only when instructed to do so. Furthermore, lines 6,7 and 9 turn the reading into a float and save it to a list for processing later on.

```

1  public static void Sensor_Reading(String receivedData, int bytesRead, InputStream
    inputStream, byte[] buffer, List dataList) throws IOException {
2
3      if (receivedData.equals("Sensor_Reading")) {
4          System.out.println("READING");
5          while (!(receivedData.equals("end_Sensor_Reading"))) {
6              bytesRead = inputStream.read(buffer);
7              receivedData = new String(buffer, 0, bytesRead);
8              try {
9                  dataList.add( Math.round(Float.parseFloat(receivedData)));
10             } catch (Exception e) {
11                 System.out.println(receivedData + ": is not a float!");
12             }
13         }
14     }
15 }

```

Listing 3.5: Java code which handles sensor readings received from the mBot

3.5 Contribution

The contribution in this section is setting up the two-way communication between the mBot and a Java program, as well as discussing different challenges encountered and identifying mitigation solutions. At this point, by encoding and defining communication, we can control the mBot and its behaviour through the bespoke firmware we built, to allow for greater flexibility in its programming.

4 | Configuring mBot with UMPLE

In this chapter, we will explore how to program the mBot to behave like a State Machine using UMPLE.

4.1 Turning the mBot into a State Machine

To treat the mBot as a State Machine, its behaviour needs to be organized in distinct States. To define the mBot's behaviours, let us set that it can only be in one active state at a time from the following: **Forward**, **Backward**, **Stop**, **Work** or **Turn** as shown in Figure 4.1.

- **Forward** implies that both DC motors move with a positive speed.
- **Backward** implies that both DC motors move with negative speed.
- **Stop** implies that both DC motors are static.
- **Turn** implies that one DC motor moves with positive speed, while the other one moves with negative speed.
- **Work** implies that the mBot turns on its LED lights in different colours.

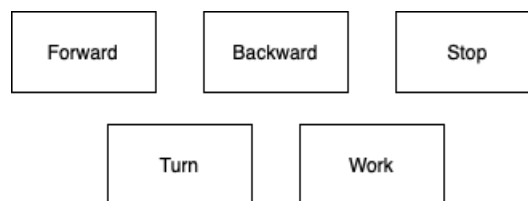


Figure 4.1: States of the mBot

These states will be time-limited, so the mBot will execute each of them for a set amount of time. This practise will be followed to avoid overloading the serial communication port, as described in section 3.4. Continuous communication would put the system at risk of malfunction.

Furthermore, we need to define transitions between states and the events that trigger them. For example, the event **obstacle** can trigger a transition between the state **Forward** to state **Turn**, in order to avoid an obstacle. Additionally, the event **doWork** can trigger a transition to the state **Work**, regardless of the previous state of the mBot.

These example transitions and events will be used to structure the UMPLE code.

4.2 UMPLE Code

Listing 4.1 shows the UMPLE code for the state machine diagram in figure 4.2, as defined in section 4.1. Within the **mBotState**, there are three state machines: **Forward**, **Turning**, and **Working**. Each state represents a distinct behaviour of the mBot. In the **Forward** state on lines

7 – 10, the robot moves forward and handles obstacle detection by transitioning to the **Turning** state and executing a turn action. Similarly, in the **Turning** state on lines 11 – 14, the robot turns and transitions back to the **Forward** state. The **Working** state on lines 15 – 19 represents an interrupt from the mBot's normal operation, where it executes a series of actions (such as stopping and beeping, represented by the function `doWork()`) before transitioning back to the **Forward** state to continue moving. Additionally, on lines 2 and 4, we are declaring that the class `mBotExecutor` extends the class `mBotCommand`. The class `mBotCommand` defines all the methods implemented by the states, such as `turn()` in `obstacle` at line 8.

```

1 namespace mBot;
2 external mBotCommand{};
3 class mBotUMPLE{
4     isA mBotCommand;
5     mBotState{
6         Forward {
7             obstacle -> / {turn();} Turn ;
8             doWork -> /{work();} Working;
9         }
10        Turn {
11            clear -> / {forward();} Forward;
12            doWork -> /{work();} Working;
13        }
14        Working {
15            doneWorking -> / {turn();} Forward;
16        }
17    }
18 }

```

Listing 4.1: UMPLE Code

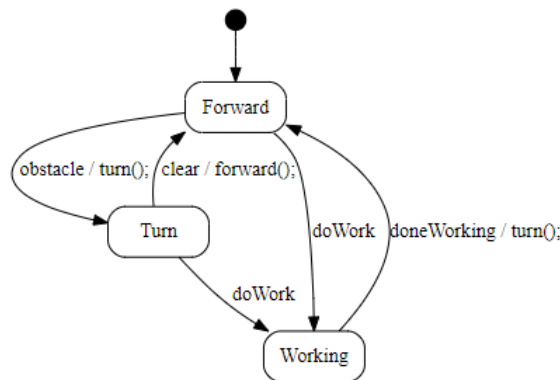


Figure 4.2: State machine diagram of listing 4.1

After we have finalised the UMPLE code shown in listing 4.1, we can generate Java code for our program from it. As mentioned in section 2.2.3, the code generated from UMPLE is not meant to be altered, and should be integrated into an existing code-base. Listing 4.2 showcases generated a code fragment regarding the `obstacle()` event. The `obstacle()` method checks the current state using a switch statement and handles obstacle detection accordingly on lines 8 – 19. In the case of the **Forward** state, it calls the `turn()` method and transitions the mBot state to **Turning**, marking the event as processed. Other states are not explicitly handled for obstacle

detection. This UMPLE-generated code demonstrates a structured approach to managing state transitions and event handling in the mBot.

```

1 public enum MBotState { Forward, Turning, Working }
2 private MBotState mBotState;
3 ...
4 public boolean obstacle()
5 {
6     boolean wasEventProcessed = false;
7
8     MBotState aMBotState = mBotState;
9     switch (aMBotState)
10    {
11        case Forward:
12            // line 11 "../model.ump"
13            turn();
14            setMBotState(MBotState.Turn);
15            wasEventProcessed = true;
16            break;
17        default:
18            // Other states do respond to this event
19    }
20
21    return wasEventProcessed;
22 }

```

Listing 4.2: Example from the UMPLE code listing 4.1 generated

We have now completed the UMPLE code and Java code generation for our state machine design. The next step is to integrate it with the firmware we built in Chapter 3 for the mBot.

4.3 UML Diagram of UMPLE Implementation

Figure 4.3 shows a UML diagram of all the code presented in sections 4.2 and 4.3. Each square represents a class. Inside the squares, the orange items represent variables of the class and the pink items represent methods of the class. The lock seen next to each item represents whether it is private or public. An open lock means that it is public, whereas a closed lock means that it is private.

For the UMPLE Implementation, three Java classes are used, `mBotCommand`, `mBotUMPLE` and `UMPLEController`. The UML diagram showcases that `mBotUMPLE` extends `mBotCommand`, as described in listing 4.1.

- `mBotCommand` defines the commands which will be sent to the mBot. It serves as an intermediate communication point between the code generated by UMPLE and the `UMPLEController`.
- `mBotUMPLE` is the generated code from UMPLE.
- `UMPLEController` handles the communication between the Java application and the C++ mBot firmware.

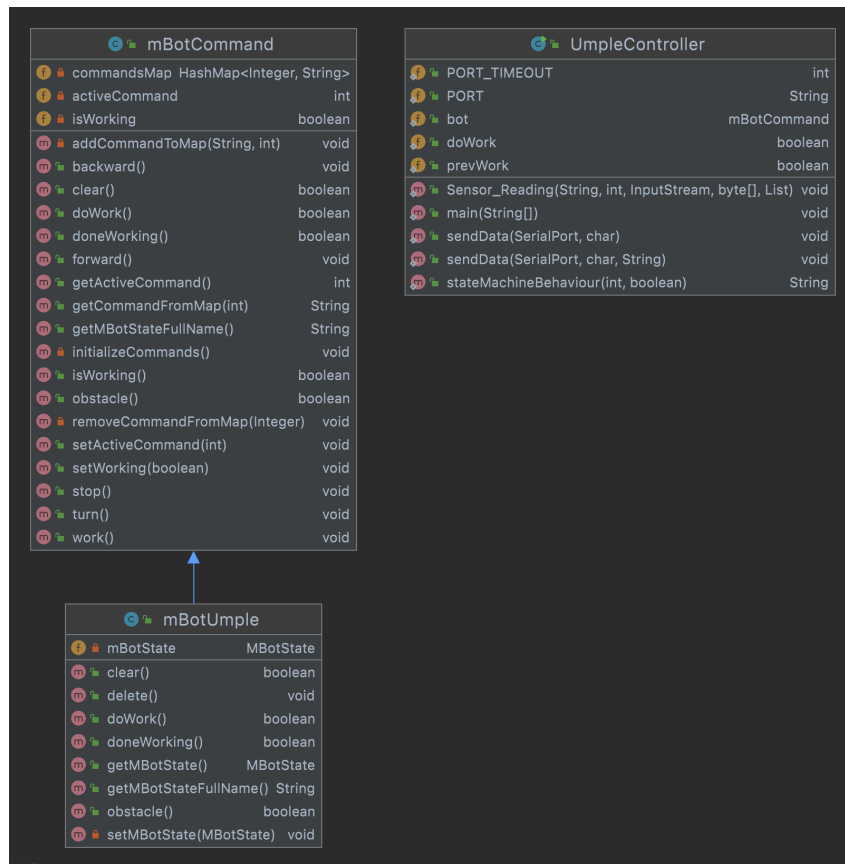


Figure 4.3: UML Diagram of UMPLE Implementation

4.4 Communication between mBot and UMPLE

The compatibility of UMPLE with mBot was a critical barrier to overcome, to establish a reliable connection. The class `mBotCommands` is designed to facilitate communication between UMPLE and the controller. A section of it is shown in listing 4.3. It contains a `HashMap` named `commandsMap` to store commands mapped to their corresponding integers, and an integer variable `activeCommand` to track the currently active command. The class defines methods such as `forward()`, `stop()`, `turn()`, `backward()`, and `work()`, each setting the `activeCommand` to a specific integer value representing different commands, such as forward movement, stop, turn, backward movement, and work-related actions respectively. These methods enable the controller to easily set and manage commands for the mBot entity, enhancing its functionality and control.

```

1 public class mBotCommand {
2
3     private HashMap<Integer, String> commandsMap;
4     private int activeCommand;
5
6     ...
7
8     public void forward(){
9         setActiveCommand(1);
10    }
11
12    public void stop(){
13        setActiveCommand(0);
14    }
15
16    public void turn(){
17        setActiveCommand(3);
18    }
19
20    public void backward(){
21        setActiveCommand(2);
22    }
23
24    public void work(){
25        setActiveCommand(4);
26    }

```

Listing 4.3: mBotCommand class to handle communication between UMPLE and the Controller

The next requires us to transmit the `activeCommand` to the mBot, so that its firmware can handle the instruction. The following code on listing 4.4 demonstrates how the controller handles communication between the mBot and UMPLE-generated code. This code follows the firmware design shown in listings 3.2 and 3.5. Line 3 declares an object of type `mBotCommand`, which is the class described in listing 4.3. Line 7 sets the termination keyword to be 'end', meaning that when the mBot transmits 'end' communication between the mBot and the Java application will end. Line 11 is the method described in listing 3.5. Line 13 retrieves the sensor reading `distance` from the list where it was saved. Line 17 implements the method that defines when events occur, the trigger transitions between states, as shown in listing 4.5. Lastly, line 19 transmits the command to the mBot, so that it can execute the instruction.

```

1 public class UMPLEController {

```

```

2    ...
3    public static mBotCommand bot = new mBotUMPLE();
4    ...
5    int bytesRead = inputStream.read(buffer);
6    String receivedData = new String(buffer, 0, bytesRead);
7    while (!(receivedData.equals("end"))) {
8        bytesRead = inputStream.read(buffer);
9        receivedData = new String(buffer, 0, bytesRead);
10
11        Sensor_Reading(receivedData, bytesRead, inputStream, buffer, dataList);
12
13        distance = Math.round(Float.parseFloat(dataList.get(0).toString()));
14        if (distance >= 100) { distance = 100;}
15
16        //outputs
17        String state = stateMachineBehaviour(distance, false);
18        char command = Integer.toString(bot.getActiveCommand()).charAt(0);
19        sendData(serialPort, command , "command: "+state);

```

Listing 4.4: Controller which handles communication between UMPLÉ and the mBot

Listing 4.5 defines when events for the state machine occur. Lines 2 – 6 check if there is an obstacle, and call the appropriate method accordingly. Lines 11 – 14 check if the mBot is working, and if so call the `doWork()` method. Lines 7 – 9 check if the mBot was previously working, and if so, they disable the function from getting repeated.

```

1    public static String stateMachineBehaviour(int distance, boolean work){
2        if (distance < 20){
3            bot.obstacle();
4        } else {
5            bot.clear();
6        }
7        if (prevWork){
8            bot.doneWorking();
9            prevWork = false;
10       }
11       if(work){
12           bot.doWork();
13           prevWork = true;
14       }
15       return bot.getMBotStateFullName();
16   }

```

Listing 4.5: Java Method to implement UMPLÉ Generated Code

Lastly, to implement the state **Work**, some adjustments had to be made in the mBot's firmware code. As shown in listing 4.6, the **Work** state is executed every 5 moves the mBot makes, in lines 4 – 9. The rest of the code is similar to the listing 3.4.

```

1    } else if (receivedChar == 'U'){
2        workCounter++;
3        delay(300);
4        if (workCounter % 5 == 0) {

```

```
5  Serial.print("doWork");
6  delay(300);
7  Serial.print("end_doWork");
8  delay(100);
9  } else {
10 Serial.print("Sensor_Reading");
11 delay(300);
12 double distance = ultraSensor.distanceCm();
13 Serial.print(distance);
14 delay(300);
15 Serial.print("end_Sensor_Reading");
16 delay(100);
17 }
```

Listing 4.6: mBot's code to determine if the state Work will be executed

4.5 Contribution

The contribution of this chapter is defining a state machine for the mBot and generating the code using UMPLE. The code developed in this chapter is fully functional and allows the mBot to avoid obstacles and perform a work operation. Therefore, successful communication between the mBot and a higher-level GenDev tool has been established.

5 | Reactive Synthesis for mBot with Spectra

Reactive systems are characterized by their ability to react to input stimuli in real-time, and as discussed in section 2.1, the mBot already has characteristics of a Reactive System. In this chapter, we will be using Spectra to elevate the behaviour of the mBot into a sophisticated Reactive System with the ability for message-driven communication.

As outlined in section 3.2, the means of communication between the mBot and a computer is serial port communication, which will be used to build message-driven communication. However, when using serial port communication, we need to be careful not to overflow the port with data. To avoid this scenario, all instructions executed by the mBot are time-limited, since continuous communication would risk overcrowding the port. Hence, after every executed instruction, the mBot will come to a stop for a set time frame, to allow for seamless communication.

5.1 mBot as a Reactive System

To model the mBot as a more complex reactive system, the focus needs to be shifted towards responding dynamically to external stimuli and events in real-time. Unlike a State Machine, a reactive system constantly monitors its environment and adapts its behaviour accordingly. Instead of redefining the sequence of movement states, the mBot's behaviours are determined in real time by sensor readings or user commands. This approach allows the mBot to dynamically change its behaviour, such as stopping or changing direction when it detects an obstacle.

In Figure 5.1 we draw the dataflow diagram corresponding to an example set of instructions that model the mBot to use its ultrasonic sensors to measure distance and act according to that measurement, as shown in Listing 3.4.

If the distance measured is too close, it can be assumed that the mBot is near an obstacle, so it turns to avoid it. If the mBot is too close to an obstacle, it goes backwards. Otherwise, it can keep going forward. The actions the mBot executes are time limited such as to not overflow the serial port communication with information. These actions will occur as long as the mBot is communicating with Spectra.

5.2 Spectra Specification

Listing 5.1 contains a detailed Spectra specification tailored to the mBot. This specification incorporates environmental data, gathered by the ultrasonic sensor, which measures distances ranging from 0 to 100 centimetres.

The mBot handles its movements via the results calculated from the Spectra specification. When distances of fewer than 10 cm are measured, which indicate an obstacle, the mBot goes backwards to avoid a collision. Whereas, distances less than 18 cm but greater than 10 cm, indicate that an obstacle is being approached, so the mBot turns and changes its direction.

For distances greater than 18 cm, the mBot moves forward, assuming the path is clear. This specification provides a basic set of instructions for the mBot to avoid collisions.

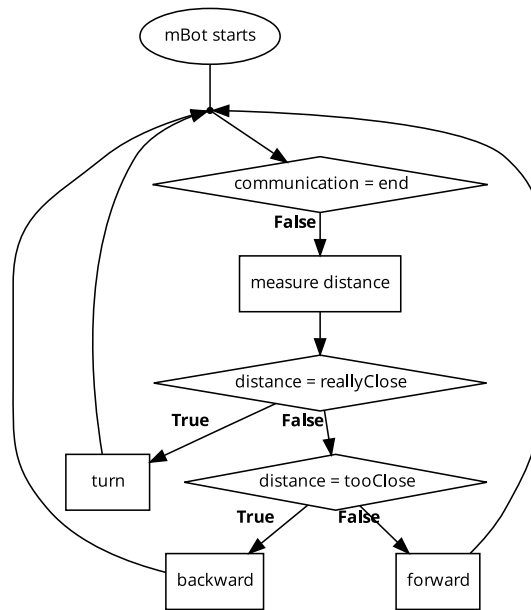


Figure 5.1: Flowchart Diagram of an example behaviour for the mBot

The result of the specification is translated into a **command** from a predefined set declared in lines 3, 7 – 9. For example, if the mBot needs to go forward, **command** will be set to 1. This result is determined in lines 15 – 17, where according to the distance measured, the keyword **implies** instructs on the next value of **command**. This command will then be delivered to the mBot via a Serial Port, prompting it to perform the action specified by the command. The commands are handled by the same code structure shown in section 4.3.

In this specific example of Spectra specification, some keywords such as **gar alw** and **define** are used. These keywords correlate to the temporal logic that Spectra is based on, as explained in section 2.2.5. The keyword **gar alw** translates to guarantee always, which means that there is a system guarantee of a safety assumption, or in other words, it is guaranteed that the assumption made will be true at some point in the system, and it is safe to assume so. (Maoz and Ringert 2021)

```

1 spec MBot_Spectra_V0
2
3 sys Int(0..3) command;
4
5 env Int(0..100) distance;
6
7 define forward := command = 1;
8 define backward := command = 2;
9 define turn := command = 3;
10
11 define tooClose := distance < 18 & distance > 10;
12 define reallyClose := distance < 10;
13 define justRight := distance > 18;
14
15 gar alw tooClose implies turn;
16 gar alw reallyClose implies backward;
17 gar alw justRight implies forward;

```

Listing 5.1: Spectra Specification for the mBot on the floor

5.3 UML Diagram of Spectra Implementation

The UML diagram in figure 5.2 showcases the class structure of the Spectra Implementation. Four Java classes are used, `mBotCommand`, `SpectraController`, `SpectraExecutor` and `CsvWriter`. The legend of the UML diagram is identical to the one presented in section 4.3.

- `mBotCommand` has the same purpose as in section 4.3, it is an intermediate communication point between Spectra and the `SpectraController`.
- `SpectraExecutor` handles the inputs and outputs of the Spectra specification shown in listing 5.1.
- `SpectraController` handles communication between the `SpectraExecutor` and the mBot C++ Firmware.
- `CsvWriter` exports the data gathered by the Ultrasonic sensors to a csv file.

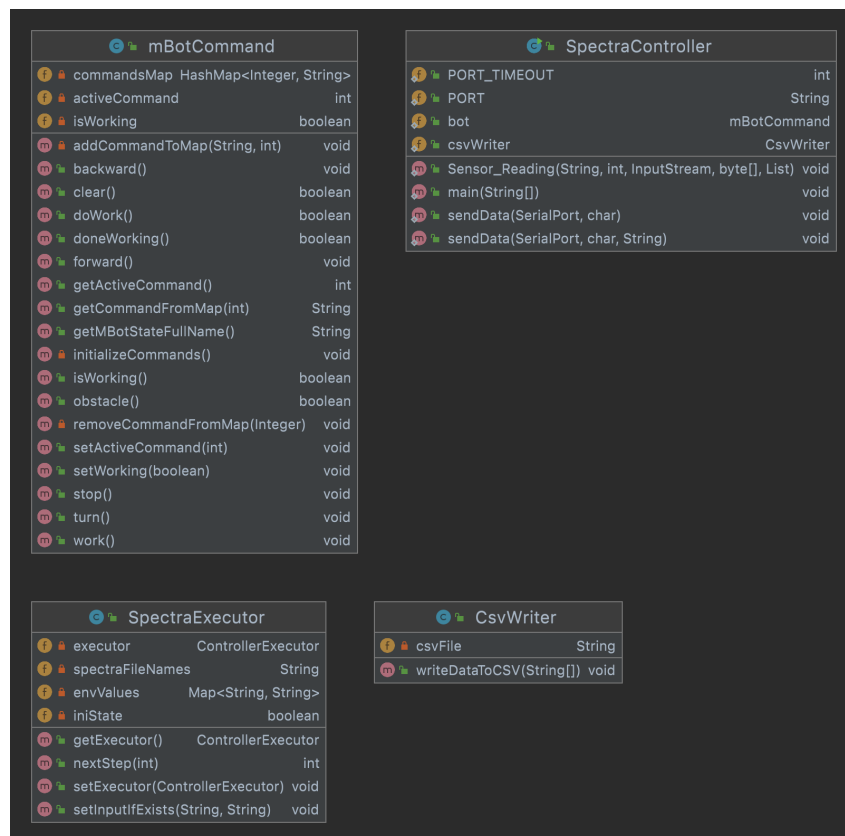


Figure 5.2: UML Diagram of the Spectra Implementation

5.4 Spectra Controller

In this section, we build the Java controllers necessary to communicate commands from the Spectra specification to the mBot.

The controller acts as an intermediate between the Spectra Specification and the mBot firmware shown in Chapter 3. It handles the Serial Port communication and the processing of the output information of the Spectra Specification.

```

1  Executor objectExecutor = new Executor("MBot_Spectra_V0");
2  ...
3  int bytesRead = inputStream.read(buffer);
4  String receivedData = new String(buffer, 0, bytesRead);
5  while (!(receivedData.equals("end"))) {
6      bytesRead = inputStream.read(buffer);
7      receivedData = new String(buffer, 0, bytesRead);
8      System.out.println("Received data: " + receivedData);
9
10     Sensor_Reading(receivedData, bytesRead, inputStream, buffer, dataList);
11
12     distance = Math.round(Float.parseFloat(dataList.get(0).toString()));
13     if (distance >= 100) { distance = 100;}
14
15     // Spectra outputs
16     int command = objectExecutor.nextStep(distance);
17     char commandCh = Integer.toString(command).charAt(0);
18     sendData(serialPort, commandCh , "command: "+commandCh);
19 }
20 ...

```

This process allows us to achieve successful transmission and reception of sensor data and commands between the mBot and our Java application.

5.5 Communication between mBot Sensors and Spectra

The next step requires us to pass the information received through listing 3.5 to the Spectra Specification. This section is part of the `SpectraExecutor` Java class.

Listing 5.2 defines a Java method named `setInputIfExists`, which is designed to handle the passing of data received from mBot sensors to the Spectra Specification. This method takes two parameters: `var`, representing a variable name, and `val`, representing the corresponding value received from the sensors. Within the method, it first checks if the specified variable exists in the environment variables managed by an executor object. If the variable exists, it attempts to add it to a map called `envValues`, associating it with the provided value. In case of any exceptions during this process, the method prints the stack trace to provide information about the error. Overall, this method ensures that data received from mBot sensors is processed and stored appropriately based on the existence of specified variables in the executor's environment.

```

1  public void setInputIfExists(String var, String val) {
2      if (executor.getEnvVars().containsKey(var)) {
3          try {
4              envValues.put(var, val);
5          } catch (Exception e) {
6              e.printStackTrace();
7          }
8      }
9  }

```

Listing 5.2: Method to pass data received from mBot sensors as to the Spectra Specification

Finally, the following code on listing 5.3 is necessary for initializing and updating the environment variables in the Spectra specification, in this case, the variable `distance`. Line 2 calls the method from listing 5.2 to set `distance`. Lines 4 – 9 check if the environment variables have been initialised, using the boolean variable `iniState`. If the executor has been initialised, then the environment variables are updated instead. The return statement on line 11 retrieves the command set by the Spectra specification for the mBot, and returns it as the next step to be followed.

```

1 public int nextStep(int distance){
2   setInputIfExists("distance", Integer.toString(distance));
3
4   if (iniState) {
5     executor.initState(envValues);
6     iniState = false;
7   } else {
8     executor.updateState(envValues);
9   }
10
11  return Integer.parseInt(executor.getCurrValue("command"));

```

Listing 5.3: Java code for the Spectra Executor to define the next command for the mBot

5.6 Evaluation of the Spectra Specification

In this section, we evaluate the behaviour of the mBot based on the Spectra specification shown in listing 5.1.

5.6.1 Navigation

Initially, the mBot's programming capabilities are limited to its proprietary IDE, mBlock, as described in Section 2.1. While mBlock correctly manages basic commands, its limitations become clear when attempting to create more complex systems that require a higher-level language.

In this section, we evaluate how using Spectra Synthesis elevates the system of the mBot, and allows for a complex system to be created. We test how message-driven communication affects the behaviour of the mBot as a Reactive System.

It is worth mentioning that the numerical values used for the Spectra Specification parameters in listing 5.1 were found by trial and error. Initially, we let the mBot roam freely within a given space while modifying the parameters until it displays behaviour that minimises collisions with obstacles. After deeper analysis, we discover some underlying logic in these numerical choices. For example, the definition of "reallyClose," where the mBot goes backwards rather than turning, refers to situations in which the distance between the mBot and the obstacle is shorter than the length of the mBot's body.

Different Environments The mBot is evaluated using testing on two different situations: ground level and elevated. This method provides a thorough evaluation of the robot's capability

under a variety of scenarios. On ground-level surfaces, the mBot's measurements of large distances ahead need to indicate a clear straight path. When navigating on elevated surfaces, the same distance measurement is crucial for identifying cliff edges and avoiding a scenario where the mBot falls. By evaluating the mBot's performance in different circumstances, we aim to obtain a better understanding of its adaptability in real-world settings.

The Spectra specification in listing 5.1 proved to be effective when navigating ground surfaces. However, when the mBot was placed on an elevated surface, such as that of a desk, it was not able to recognise a cliff.

Hence, a new specification based on 5.1 was created to accommodate for this flaw, shown in listing 5.4. A new variable is defined, `tooFar`, which indicates that the distance measured could potentially be a cliff. Therefore, now the mBot will go backwards if it detects a distance that is either `reallyClose` or `tooFar`. This new specification allows the mBot to function on an elevated surface, but if used back on the ground it could cause it to go backwards needlessly.

```

1 spec MBot_Spectra_V1
2
3 sys Int(0..3) activeCommand;
4
5 env Int(0..100) distance;
6
7 define forward := activeCommand = 1;
8 define backward := activeCommand = 2;
9 define turn := activeCommand = 3;
10
11 define tooClose := distance < 18 & distance > 10;
12 define reallyClose := distance < 10;
13 define tooFar := distance > 45;
14 define justRight := distance > 18 & distance < 45;
15
16 gar alw tooClose implies turn;
17 gar alw reallyClose | tooFar implies backward;
18 gar alw justRight implies forward;

```

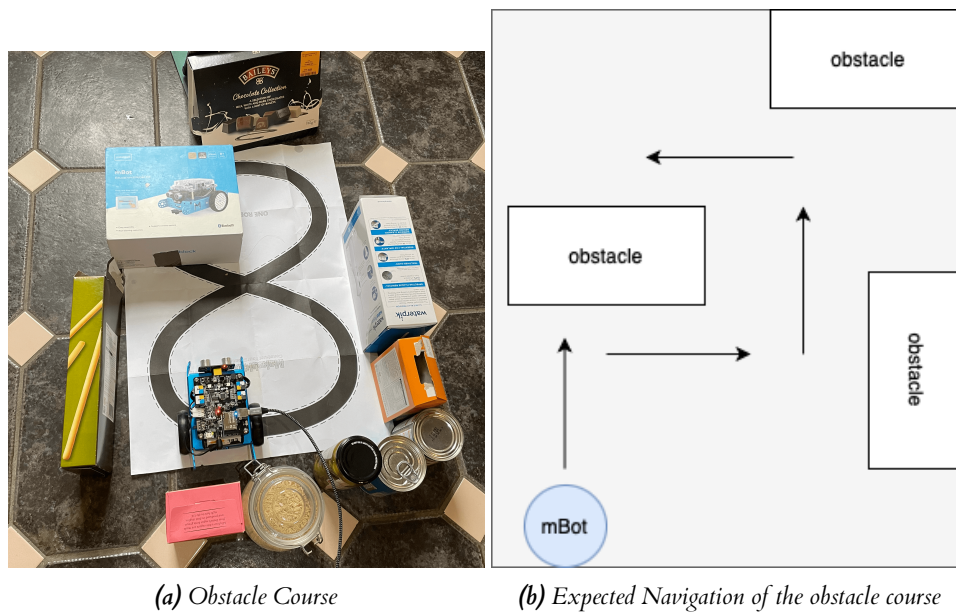
Listing 5.4: Spectra Specification for elevated surfaces

Obstacle Course In this experiment, the mBot needs to manoeuvre its way through an obstacle course. The purpose of this experiment is to observe the behaviour of the reactive system when in a more confined space, in contrast to the previous experiment, where the mBot navigated in an open space with less obstacles. Figure 5.3 shows the obstacle course and the expected navigation of it.

A video of the mBot navigating the obstacle course can be found here: <https://www.youtube.com/shorts/S-VI37uYWyo>.

In the confined space of our obstacle course, the mBot demonstrates satisfactory navigation skills, able to avoid most obstacles, but not all. A recognised limitation is the lack of a mechanism for the mBot to determine whether turning left or right is the more efficient way to navigate around obstacles. This flaw limits its ability to optimise its path through the course.

The mBot's rear end is wider than its front-facing ultrasonic sensor, which contributes to the problem of improper navigation of confined spaces. The sensor's limited range fails to cover the entire width of the mBot, leaving blind spots where the mBot may collide accidentally. For example, when faced with closely spaced obstacles, the mBot may choose to go backwards to



avoid a front collision, only to unintentionally collide with an obstacle behind it. This scenario demonstrates an important vulnerability in the mBot's current sensor configuration.

Integrating additional sensors on the mBot's sides could address both of these flaws. By equipping the mBot with sensors that can measure distance from its back, right, and left sides, it can gain a comprehensive understanding of its surroundings. This improved sensory input would enable the mBot to make more informed navigational decisions, lowering the risks posed by blind spots and allowing it to navigate the obstacle course with greater precision and efficiency.

5.6.2 Sensor Output and mBot Response

Sensor outputs for a ground surface are shown on Table 5.1, compared to commands executed by the mBot:

5.7 Contribution

This chapter makes a contribution by designing a Spectra specification for the mBot, allowing it to evolve into a complex reactive system. The communication between Spectra and the mBot is successful, paving the way for the use of temporal logic in educational robotics programming.

Table 5.1: *Data Table for a Ground Surface*

Distance	Command
100	Forward
100	Forward
100	Forward
100	Forward
100	Forward
100	Forward
7	Backwards
4	Backwards
12	Turn
100	Forward
16	Turn
25	Forward
5	Backwards

6 | Initial Steps in WiFi Module Integration

In this chapter, we begin the process of integrating a third-party Wi-Fi module with the mBot platform. This integration has the potential to develop seamless communication channels, bypassing the limitations imposed by standard serial port connections. By implementing this feature, we aim to increase the range and flexibility of the mBot system and allow for complex systems to be tested.

The design presented in this chapter was inspired by a post in the mBot forums trying to achieve a similar goal (MakeblockForum 2017). However, the information presented contains inaccuracies and inconsistencies. We took this post into consideration while designing a detailed method to achieve a successful connection between the mBot and the ESP8266 D1 Mini Wi-Fi Module by Wemos.

6.1 Circuit Schematic

6.1.1 ESP8226 - D1 Mini Pin Diagram

Below, a Pinout of the ESP8266 D1 Mini by Wemos is shown. This board is small enough to be able to be fitted on the mBot. The only pins that matter for the connection are GND, 3.3V, RX and TX. Great precaution should be taken when connecting the 3.3V pin to the mBot's 5V output. If a voltage divider is not used, as shown in subsection 6.1.2, there is a chance that the ESP8266 might become damaged. In this case, a new D1 Mini should be used, as the previous one will not be functional. The TX pin is where digital data is being transmitted, whereas the RX pin is where digital data is being received.

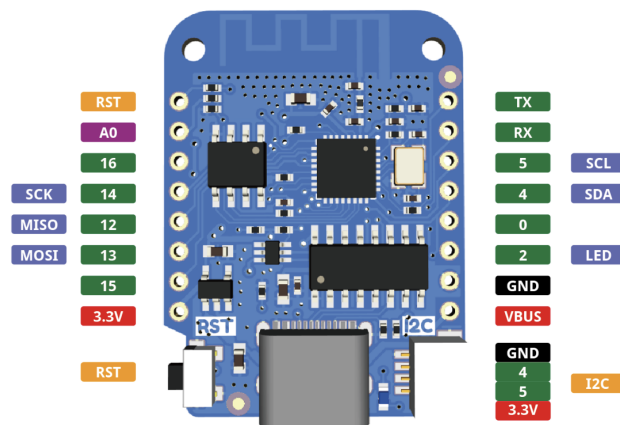


Figure 6.1: ESP8226 - D1 Mini Pinout (wemos.cc 2021)

6.1.2 Voltage Divider

A voltage divider is a circuit consisting of two resistors connected in series across a voltage source. It divides the input voltage into a lower output voltage based on the ratio of the two resistor values. It's commonly used to provide a specific voltage level for powering components in electronic circuits, in this case, to connect the ESP8226 D1 Mini to the mCore. The following is the standard equation for a voltage divider:

$$V_{out} = V_{in} \times \frac{R_2}{R_1 + R_2} \quad (6.1)$$

Where:

- V_{out} is the Voltage at the output of the divider,
- V_{in} the Voltage at the input of the divider,
- R_1 is the resistance of the first resistor in the series,
- and R_2 is the resistance of the second resistor in the series.

Given:

$$\begin{aligned} V_{out} &= 3.3 \text{ V} \\ V_{in} &= 5 \text{ V} \\ R_1 &= 10 \text{ k}\Omega \end{aligned}$$

We can solve equation 6.1 for R_2 :

$$\begin{aligned} 3.3 &= 5 \times \frac{R_2}{10\text{k} + R_2} \\ 3.3(10\text{k} + R_2) &= 5 \times R_2 \\ 33\text{k} + 3.3R_2 &= 5R_2 \\ 33\text{k} &= 1.7R_2 \\ R_2 &= \frac{33\text{k}}{1.7} \approx 19411 \Omega \approx 20\text{k} \Omega \end{aligned}$$

According to these calculations, we can draw the figure 6.2. It should be noted that the value of R_1 is arbitrary, in this project $10\text{k}\Omega$ is used because it is a commonly accessible value in resistors.

In the circuit schematic in figure 6.2, the proposed layout regarding the connection between the ESP8226 D1 Mini and the mCore is shown. The ground (GND) pin of the D1 Mini should be connected to the GND pin of the mCore, and the corresponding TX and RX pins should be connected to each other as instructed. The most crucial part of the circuit is the voltage divider.

6.2 Software Implementation

Now, assuming that the ESP8226 has been connected successfully, establishing Wi-Fi communication instead of Serial Port communication is feasible. The only thing left is to alter the firmware to handle Wifi communication. To do this, the following libraries can be used, as shown by Professor Oliver Jan Ringert in GenDevBot (Ringert 2023):

- **ESP8266WiFi.h** used to initiate a connection to the ESP8266 from the mBot Firmware
- **ESP8266HTTPClient.h** used to manage HTTP requests from the ESP8266
- **WiFiManager.h** used to manage dynamic connection to Wi-Fi networks, without the need for hardcoding credentials.

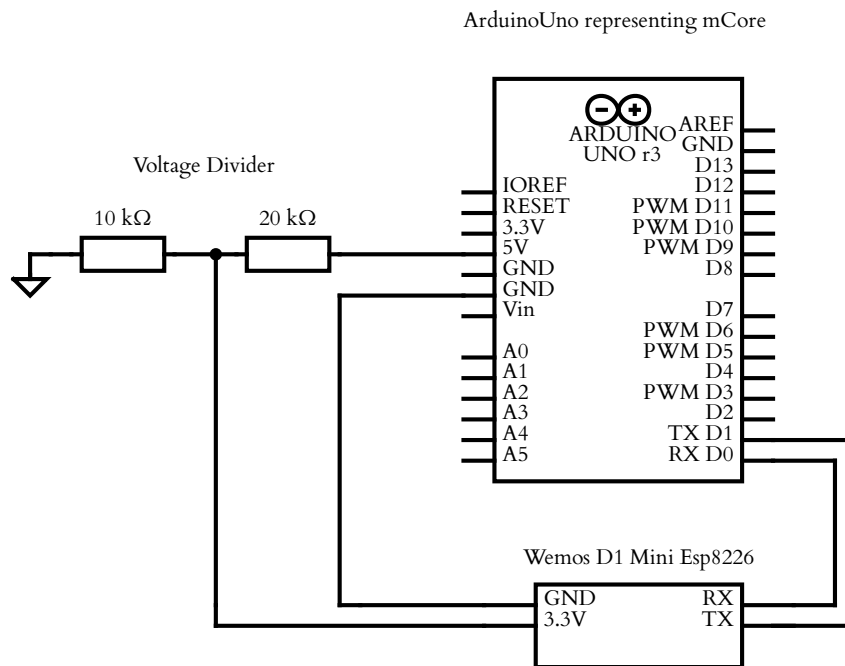


Figure 6.2: Schematic for connecting the ESP8226 D1 Mini to the mCore

6.3 Contribution

The contribution of this chapter is the presentation of the initial steps to take to integrate the ESP 8266 module into the mBot. The purpose of this chapter is to explain in detail the future development of the mBot's hardware, for anyone wishing to take this step further.

7 | Further Discussion

In this chapter, we conduct a thorough evaluation of the concept of using higher-level generative development tools on educational robots.

7.1 Conceptual Evaluation

Throughout this project, we aim to examine the concept of integration of higher-level infrastructure with educational robots, with the mBot serving as a proof of concept to demonstrate the feasibility of the project. While acknowledging the existing possibility of generative development (GenDev) on the mBot, we recognize that the infrastructure provided by Makeblock is restrictive. Through the utilization of UMPLE and Spectra, we aim to overcome these restrictions and provide a more flexible and powerful framework for GenDev on the mBot. The successful establishment of connections between the mBot, UMPLE, and Spectra highlight the potential of integrating abstract concepts with physical robotics.

UMPLE proves effective in managing state machines. However since state machines have a predefined amount of transitions, there are limitations in scenarios involving navigation in unknown environments. Keeping in mind that this project focuses on educational robots, we can assume that unknown environments will be a common occurrence in educational settings. In contrast, Spectra Synthesis, concerning reactive systems, showcases greater adaptability and effectiveness in navigating unknown environments.

While building the framework for the mBot, compatibility with other Arduino Uno-based systems is taken into account by using provided libraries for the Arduino Uno platform. Nonetheless, the reliance on Makeblock's proprietary library for certain sensors of the mBot raises concerns regarding compatibility with other educational robots, thereby highlighting a potential limitation of our approach.

Lastly, the framework we build is based on serial port communication. This method of communication restrains the space where the mBot is able to move in. We address this limitation in chapter 6 regarding the integration of a Wi-Fi module. Additionally, sophisticated object-oriented design patterns can be implemented in our design, to allow the code to be expandable.

7.2 Challenges

7.2.1 Working with the mBot Hardware

The ready-to-use setup of the mBot saves time and offers a smooth learning experience for users. However, the pre-defined hardware configuration makes it difficult to add new components or functionalities, resulting in limited expandability.

For example, while investigating the prospect of enabling Wi-Fi connectivity with a computer, it is discovered that the mBot lacks a built-in Wi-Fi module. Although a version of the mBot that includes a WiFi module exists, the fact that the model is no longer in production rules out the idea. Another possibility is to integrate third-party hardware, such as the ESP8266.

However, this approach creates additional challenges, such as uncertainty of compatibility with the mCore firmware and the possibility of harming the existing system. While technically feasible, compatibility issues and the necessity for major adjustments highlight the difficulty of expanding the mBot's capabilities beyond the pre-configured arrangement.

Therefore, there is a decision to be made between using pre-assembled robot kits such as the mBot versus purchasing individual components to create a customised robot. While pre-assembled kits are convenient and easy to use, they may have limited adaptability and customisation choices. In contrast, constructing a robot from individual components provides greater flexibility and control over the final design, but necessitates more technical knowledge and time investment.

7.2.2 Working with Makeblock Proprietary Software

Throughout this project, the challenge posed by the proprietary software within the mBot is repeatedly highlighted, emphasising the challenges of integrating off-the-shelf educational robots with external technologies. Makeblock has carefully developed the mBot's software to work seamlessly with their proprietary IDE, mBlock. However, this design makes it difficult to connect the mBot to different technologies or programming environments. One notable example of this challenge, as discussed in Chapter 3, is the inaccessibility of the mBot's Bluetooth module outside of mBlock. The software architecture on the mBot's board appears to limit the recognition of the Bluetooth module by other software or devices, making a critical component of the mBot inaccessible for external communication during this project.

To progress with our research goals, this technical difficulty needs to be resolved. Initially, locating the drivers needed to enable the mBot's visibility to external machines proved difficult and requires extensive web research. Furthermore, the lack of user-friendly tutorials for the Makeblock libraries used to control the mBot via C++ in ArduinoIDE presents an additional challenge. Due to a lack of user-friendly tutorials, the source code on Makeblock's GitHub repository is inspected in order to understand how these libraries work.

Overall, this experience sheds light on the limited support offered by Makeblock for users seeking to extend the functionality of their hardware. This raises important questions about the advertised versatility of off-the-shelf educational robots like the mBot. The disparity between the advertised versatility and the practical limitations encountered emphasises the need for greater transparency and accessibility in the development and support processes of educational robotics platforms.

8 | Conclusion

8.1 Summary

The aim of this project was to explore the use of high-level GenDev tools within the context of educational off-the-shelf robots, with a particular focus on the mBot from Makeblock. Despite being designed for educational purposes, the mBot's programming options are restricted to Makeblock's proprietary software, limiting its flexibility. This limitation hinders more complex software developments such as high-level GenDev tools from being utilized with the mBot.

However, through the development of a robust framework encompassing C++ Firmware and Java Controllers, communication and programming capabilities with Spectra and UMPLE were successfully established, laying the groundwork for potential integration with additional technologies. Chapter 4 delves into the details of implementing UMPLE, enabling the creation of state machine behaviour within the mBot, while Chapter 5 focuses on the integration and utilization of Spectra to develop a more sophisticated reactive system.

Serial port communication was chosen for its widespread availability and simplicity, although wireless communication would offer greater freedom and efficiency. Although wireless communication wasn't achieved due to limitations with Makeblock's hardware, exploration into enabling wireless communication by integrating third-party WiFi modules is discussed in Chapter 6, while highlighting potential hardware complications.

In Chapter 7, the project's success is evaluated, considering the feasibility and challenges of utilizing higher-level GenDev tools with off-the-shelf robots UMPLE and Spectra were evaluated both theoretically, with Spectra being further evaluated in practical experiments.

8.2 Reflection

Upon reflection, my project journey took a different course from its initial proposal, which aimed to explore implementing GenDev logic on the mBot from a theoretical approach. Early on, it became clear that there was a critical missing piece: a framework for connecting UMPLE, Spectra, and the mBot. Applying my background in Electronic and Software Engineering, I set out to engineer a solution, drawing from knowledge acquired in courses like Networked Systems 3, Communication Systems 3, and Realtime Systems 3.

I quickly realised that not only did a framework not exist, but it was also challenging to develop since Makeblock, the company that created the mBot, imposed restrictions that limited the versatility of the Arduino Uno-based mCore board. By looking into this matter further, I realised that many of the imposed restrictions, such as the inability to easily install a third-party wifi module, were sold as expensive add-ons by Makeblock.

Despite the potential for higher abstraction software and hardware to improve robotics learning for beginners, these restrictions raise concerns about whether they were intended to increase profit, hindering the creative process behind educational robots. The choice to restrict development to Makeblock's own software could be justified as protecting products from poor quality or malicious third party software from being used. However, limiting further expansion and development for

an educational product meant to be an introduction to robotics and computer science, such as the mBot, could be seen as antithetical to the objective of making such products.

8.3 Future Work

If we were to continue this project, there are a number of tasks that would aid in the progression of the stated aims:

1. Investigate improving the structure of the developed Java framework through the use of Object Oriented Design principles to make the system more expandable.
2. Further explore wireless connection using the Wi-Fi Module shown in Chapter 6
3. Investigate the possibility of including multiple ultrasonic sensors to make the mBot more aware of its environment
4. Investigate how to enable the robot to decide on whether to turn left or right, instead of having a generic turn functionality

A | Appendices

A.1 mCore PCB Schematic

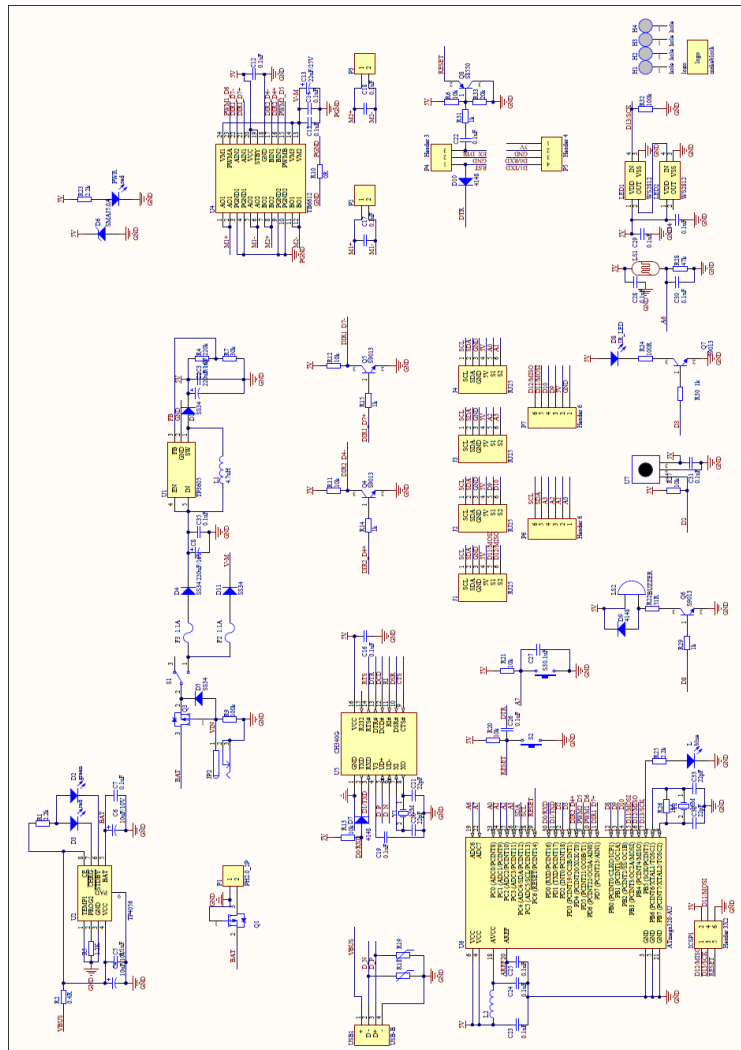


Figure A.1: mCore PCB Schematic

A.2 Complete UML Diagram of the Java Application

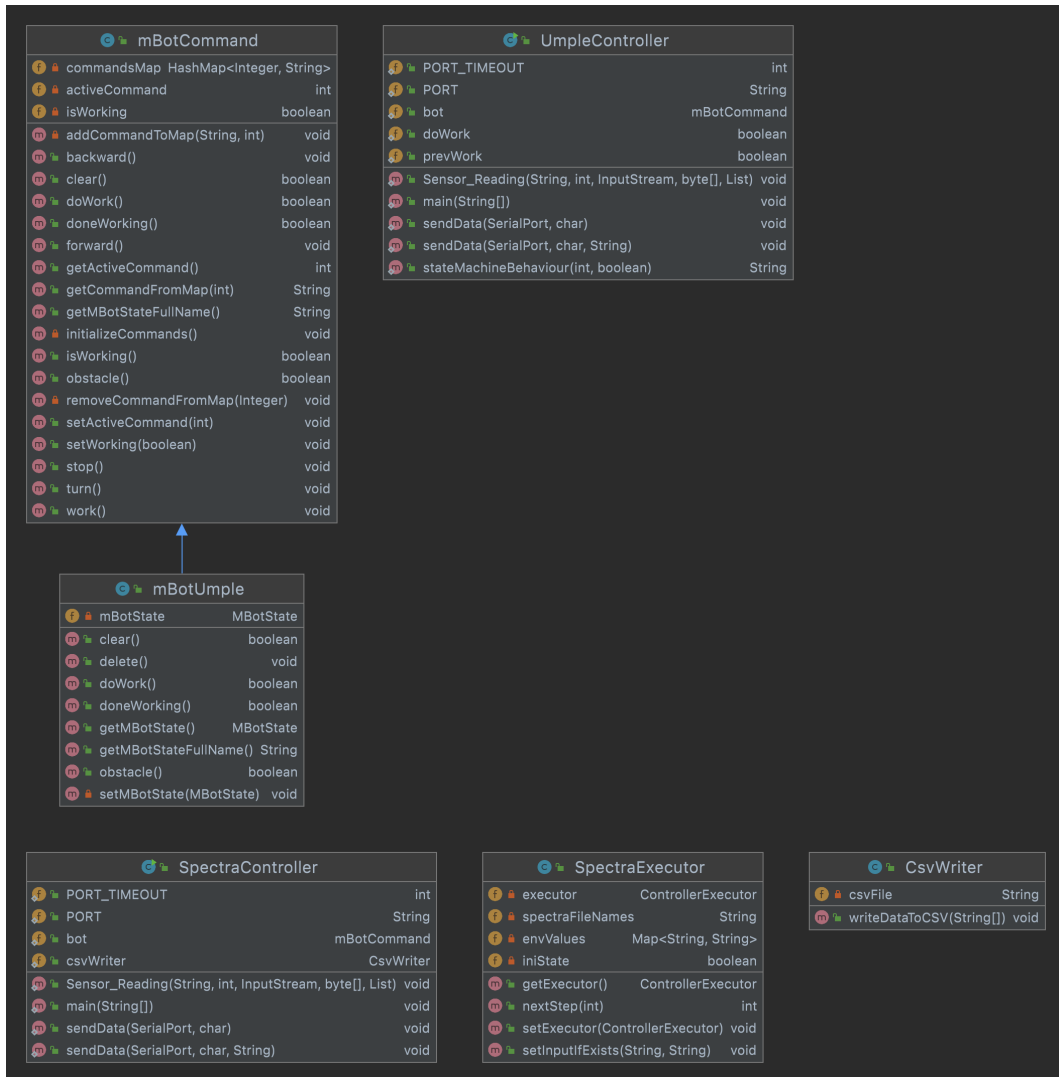


Figure A.2: Complete UML Diagram of the Java Application

Bibliography

- C. Baier and J.-P. Katoen. *Principles of Model Checking*. The MIT Press, 2008.
- M. Brice and et al. Jarduino. <https://github.com/SINTEF-9012/JArduino>, 2018.
- K. Czarnecki. Overview of Generative Software Development. In J.-P. Banâtre, P. Fradet, J.-L. Giavitto, and O. Michel, editors, *Unconventional Programming Paradigms*, Lecture Notes in Computer Science, pages 326–341, Berlin, Heidelberg, 2005. Springer. ISBN 9783540314820. doi: 10.1007/11527800_25.
- L. Frenzel. *Handbook of Serial Communications Interfaces: A Comprehensive Compendium of Serial Digital Input/Output (I/O) Standards*. 2016. URL <https://www.sciencedirect.com/book/9780128006290/handbook-of-serial-communications-interfaces#book-info>.
- T. C. Lethbridge, A. Forward, O. Badreddin, D. Brestovansky, M. Garzon, H. Aljamaan, S. Eid, A. Hussein Orabi, M. Hussein Orabi, V. Abdelzad, O. Adesina, A. Alghamdi, A. Algablan, and A. Zakariapour. Umple: Model-driven development for open source and education. *Science of Computer Programming*, 208:102665, Aug. 2021. ISSN 0167-6423. doi: 10.1016/j.scico.2021.102665. URL <https://www.sciencedirect.com/science/article/pii/S0167642321000587>.
- Makeblock. Makeblock libraries: Arduino library for makeblock electronic modules. <https://github.com/Makeblock-official/Makeblock-Libraries>, 2020.
- Makeblock. mCore – Main Control Board of mBot, Nov. 2021. URL <https://support.makeblock.com/hc/en-us/articles/4412894402967-mCore-Main-Control-Board-of-mBot>.
- MakeblockForum. Wifi (esp8266) robot. Online forum post, 2017. URL <https://forum.makeblock.com/t/wifi-esp8266-robot/7342>.
- S. Maoz and J. O. Ringert. Spectra: A specification language for reactive systems. *Software and Systems Modeling*, 20(4):1553–1586, 2021. doi: 10.1007/s10270-021-00868-z.
- Oracle. Java Communications API. URL <https://www.oracle.com/java/technologies/java-communications-api.html>.
- J. Pisarov. Experience with mbot – wheeled mobile robot. volume Proceedings of the XXXV. Jubileumi Kandó Konferencia 2019 (JKK2019), 11 2019.
- J. O. Ringert. [a1] synthesize your first controller (solution), 2021. URL <https://www.youtube.com/watch?v=6yG0y4q9Zpw>. Video unavailable in some regions.
- J. O. Ringert. gendev.bot: Public materials for small rover robot used in generative development module. <https://github.com/jringert/gendev.bot>, 2023.

- P. Royal. What Is a Reactive System? In P. Royal, editor, *Building Modern Business Applications: Reactive Cloud Architecture for Java, Spring, and PostgreSQL*, pages 23–29. Apress, Berkeley, CA, 2023. ISBN 9781484289921. doi: 10.1007/978-1-4842-8992-1_3. URL https://doi.org/10.1007/978-1-4842-8992-1_3.
- M. Shead. Understanding state machines. *freeCodeCamp.org*, February 2018. URL <https://www.freecodecamp.org/news/state-machines-basics-of-computer-science-d42855debc66/>.
- M. C. S. Team. Program mbot in arduino ide. <https://support.makeblock.com/hc/en-us/articles/4419572961943-Program-mBot-in-Arduino-IDE>, 2022.
- UMPLE. Umple: Merging Modeling with Programming. URL <https://cruise.umple.org/umple/>.
- wemos.cc. LOLIN D1 mini — WEMOS documentation, 2021. URL https://www.wemos.cc/en/latest/d1/d1_mini.html.
- V. Yodaiken. State machines for large scale computer software and systems. *Formal Aspects of Computing*, Nov. 2023. ISSN 0934-5043. doi: 10.1145/3633786. URL <https://dl.acm.org/doi/10.1145/3633786>. Just Accepted.